

系统接口自动化测试方案

1. 环境要求

- **操作系统:** Windows 10/11 / Linux
- **Python:** 3.6或更高版本
- **Java:** JDK 8+ (Allure 依赖)
- **Git:** 版本管理

2. 目标和范围

- **目标:**
 - 确保接口功能正确性、稳定性和可靠性。
 - 快速回归，防止新代码引入缺陷。
 - 验证系统边界条件和异常处理能力。
- **范围:**
 - 根据 实际给到的接口 编写对应的测试用例

3. 测试策略与架构

- **技术选型:**
 - 语言: Python
 - 测试框架: Pytest
 - HTTP客户端: Requests请求库
 - 断言库: Pytest内置断言
 - 数据管理: json/yaml/csv文件, 或pytest的@pytest.mark.parametrize 实现参数化。
 - 报告生成: Allure

- **架构设计:**

接口测试脚手架仓库地址: https://gitee.com/SuLiu_Laile/api_test_demo.git

1. 配置文件 (config)

- 编写一些通用的配置

示例结构:

```
config/
├─ __init__.py
├─ setting.ini          # 主配置文件
├─ dev_config.yaml     # 开发环境配置
├─ test_config.yaml    # 测试环境配置
└─ prod_config.yaml    # 生产环境配置
```

示例文件: config/setting.ini

`setting.ini`

```

;主机地址
[host]
api_sit_url = http://127.0.0.1:8080

;数据库配置
[mysql]
MYSQL_HOST=127.0.0.1
MYSQL_PORT=3306
MYSQL_USER=admin
MYSQL_PASSWD=123456
MYSQL_DB=test_db

```

2. 请求方法封装 (core)

- requests请求方法封装、关键字返回结果类

示例结构:

```

core/
├─ __init__.py
├─ rest_client.py    # 封装requests库的GET/POST/PUT/DELETE方法
└─ ApiService.py    # 用例执行的核心流程

```

示例文件: core/ApiService.py

```

import json

import requests

from utils.log_util import logger
from utils.read import base_data

api_base_url = base_data.read_ini()['host']['api_site_url'] # 接口基
地址

class RestClient:
    def __init__(self):
        self.api_base_url = api_base_url
        self.session = requests.Session()

    def do_request(self, url, method, **kwargs):
        response = self.request(url, method, **kwargs).json()
        logger.info("接口返回内容>>>\n{}".format(json.dumps(response,
ensure_ascii=False, indent=2)))
        return response

    def request(self, url, method, **kwargs):
        self.request_log(url, method, **kwargs)
        if method == "GET":
            return self.session.get(self.api_base_url +
url, **kwargs)
        if method == "POST":
            return self.session.post(self.api_base_url +
url, **kwargs)
        if method == "PUT":
            return self.session.put(self.api_base_url +
url, **kwargs)

```

```

        if method == "DELETE":
            return self.session.delete(self.api_base_url +
url,**kwargs)
    def request_log(self, url, method, **kwargs):
        data = dict(**kwargs).get('data')
        json_data = dict(**kwargs).get('json')
        params = dict(**kwargs).get('params')
        headers = dict(**kwargs).get('headers')

        logger.info("接口请求的地址>>>{}".format(self.api_base_url +
url))
        logger.info("接口请求的方法>>>{}".format(method))
        if data is not None:
            logger.info("接口请求的data参数>>>
{}".format(json.dumps(data,ensure_ascii=False,indent=2)))
            if json_data is not None:
                logger.info("接口请求的json参数>>>
{}".format(json.dumps(json_data,ensure_ascii=False,indent=2)))
            if params is not None:
                logger.info("接口请求的params参数>>>
{}".format(json.dumps(params,ensure_ascii=False,indent=2)))
            if headers is not None:
                logger.info("接口请求的headers参数>>>
{}".format(json.dumps(headers,ensure_ascii=False,indent=2)))

api_util = RestClient()

```

3. 测试用例数据 (data)

- 存放测试数据

示例结构：

```

data/
└─ user_data.yaml      # 用户管理测试数据

```

示例文件：

```

user_login:
    # 接口的基本信息
    - request_info:
        case_title: 登录接口
        url: /member/auth/email-login
        method: POST
        headers:
            Content-Type: application/json
    case_info:
        # 接口请求参数, params, json, data
        - json:
            account: 2594359191@qq.com
            captchaKey: ''
            captchaVerification: ''
            code: ''
            email: ''
            isDistributor: 'true'
            newPassword: ''
            password: Cf@2000...

```

```

        scene: 1
        username: ''
    validate:
        - eq: [ $.data.userId, '901' ]
    extract:
        token: $.data.userId # 提取用户登录的token
        sdasda: 12131

- json:
    account: 2594359199@qq.com
    captchaKey: ''
    captchaVerification: ''
    code: ''
    email: ''
    isDistributor: "true"
    newPassword: ''
    password: Cf@2000...
    scene: 1
    username: ''
    validate:
        - eq: [ $.msg, '未注册' ]

```

4. 测试报告 (report)

- 运行完毕之后生成的测试报告

5. 测试用例 (testcases)

- 存放所需要的测试用例

示例结构:

```

testcases/
├─ __init__.py
├─ conftest.py # pytest配置和fixture
├─ test_user.py # 用户管理测试用例
└─ test_file_operations.py # 文件操作测试用例

```

示例结构: test_user_management.py

```

import allure
import pytest

from core.ApiService import ApiService
from utils.YamlUtil import YamlUtil

@allure.feature("用户模块")
class TestUser:

    @pytest.mark.parametrize("data", YamlUtil().extract_case("user.yaml", "user_login"))
    def test_user(self, data):
        ApiService().handle_case(data)

```

6. 工具层 (utils)

- 存放一些自定义工具

示例结构:

```
utils/  
├─ __init__.py  
├─ AssertUtil.py      # 断言工具  
├─ YamlUtil.py        # yaml文件操作工具  
├─ response_util.py   # 结果处理工具  
├─ read.py            # 配置文件读取工具  
└─ ExtractUtil.py     # 结果存放工具
```

示例文件: utils/Assertutil.py

```
import jsonpath  
  
class AssertUtil:  
  
    @staticmethod  
    def equals(check_value, expect_value):  
        """相等"""  
        assert check_value == expect_value, f'{check_value} ==  
{expect_value}'  
  
    @staticmethod  
    def less_than(check_value, expect_value):  
        """小于"""  
        assert check_value < expect_value, f'{check_value} <  
{expect_value}'  
  
    @staticmethod  
    def less_than_or_equals(check_value, expect_value):  
        """小于等于"""  
        assert check_value <= expect_value, f'{check_value} <=  
{expect_value}'  
  
    @staticmethod  
    def greater_than(check_value, expect_value):  
        """大于"""  
        assert check_value > expect_value, f'{check_value} >  
{expect_value}'  
  
    @staticmethod  
    def greater_than_or_equals(check_value, expect_value):  
        """大于等于"""  
        assert check_value >= expect_value, f'{check_value} >=  
{expect_value}'  
  
    @staticmethod  
    def not_equals(check_value, expect_value):  
        """不等于"""  
        assert check_value != expect_value, f'{check_value} !=  
{expect_value}'  
  
    @staticmethod
```

```

def contains(check_value, expect_value):
    """包含"""
    assert expect_value in check_value, f'{expect_value} in {check_value}'

    @staticmethod
    def startswith(check_value, expect_value):
        """以什么开头"""
        assert str(check_value).startswith(str(expect_value)),
            f'{str(check_value)} startswith {str(expect_value)}'

    @staticmethod
    def endswith(check_value, expect_value):
        """以什么结尾"""
        assert str(check_value).endswith(str(expect_value)),
            f'{str(check_value)} endswith {str(expect_value)}'

    @staticmethod
    def length(check_value, expect_value):
        """校验数量"""
        assert len(check_value) == expect_value, f'{str(check_value)}
        == {str(expect_value)}'

    def extract_by_jsonpath(self, extract_value: dict,
        extract_expression: str): # noqa
        """
            json path 取值
            :param extract_value: response.json()
            :param extract_expression: eg: '$.code'
            :return: None或 提取的第一个值 或全部
        """
        if not isinstance(extract_expression, str):
            return extract_expression
        extract_value = jsonpath.jsonpath(extract_value,
        extract_expression)
        if not extract_value:
            return
        elif len(extract_value) == 1:
            return extract_value[0]
        else:
            return extract_value

    def validate_response(self, response, validate_check):
        """校验结果"""
        for check in validate_check:
            for check_type, check_value in check.items():
                # 实际结果
                actual_value = self.extract_by_jsonpath(response,
                check_value[0])
                # 期望结果
                expect_value = check_value[1]
                if check_type in ["eq", "equals", "equal"]:
                    self.equals(actual_value, expect_value)
                elif check_type in ["lt", "less_than"]:
                    self.less_than(actual_value, expect_value)
                elif check_type in ["le", "less_or_equals"]:
                    self.less_than_or_equals(actual_value,
                    expect_value)

```

```
elif check_type in ["gt", "greater_than"]:
    self.greater_than(actual_value, expect_value)
elif check_type in ["ne", "not_equal"]:
    self.not_equals(actual_value, expect_value)
elif check_type in ["contains"]:
    self.contains(actual_value, expect_value)
elif check_type in ["startswith"]:
    self.startswith(actual_value, expect_value)
elif check_type in ["endswith"]:
    self.endswith(actual_value, expect_value)
elif check_type in ["length"]:
    self.length(actual_value, expect_value)
else:
    print(f'{check_type} not valid check type')
```

4. 执行策略与CI/CD集成

- 测试分类:
 - 回归测试：全量测试用例，每晚定时执行。
- CI/CD集成:
 - 触发条件：每天定时、主干分支每次提交后。
 - 流程： 代码编译 -> 环境部署 -> 执行自动化测试 -> 生成测试报告